Tadashi

A field manual for the working environment

RITESH ROSHAN

October 2026

Version 1.1

Contents

	Before you begin	1
1	How it fits together	2
	The folder	3
	A new machine	3
2	The command line	4
	Search by structure	4
	Measure before you claim	5
	The shell	5
3	The agents	6
	Standing orders	6
	Memory	6
	Working in parallel	7
4	Skills	8
	The active set	8
	The library	9
5	Documents with folio	11
	Writing the source	11
	Building	11
	LaTeX and Office files	12
	Writing Typst directly	12
6	Starting something new	13
7	Intelligence	14
	The toolkit	14
	Method	14
8	Privacy	16
	What changed	16
	Trusted applications	17
	Watching the machine	17
	Disk encryption	17
9	The arsenal	19
	Typefaces	19
	Interfaces	19
	References worth owning	20
	Habits	20
10	Keeping it sharp	21
11	Tool reference	22

Before you begin

Tadashi is the name of this environment and of the folder that rebuilds it. It holds the command-line tools that make a Windows machine fast, the standing orders that every Claude session follows, the skills that turn recurring work into a single instruction, an intelligence toolkit, and the privacy settings that keep the machine yours. Everything lives in one directory, so a new machine reaches the same state with three commands.

This manual describes what is installed, why each piece earned its place, and how to keep the whole thing sharp. It was typeset by the folio skill it documents, which makes the document its own first test.

CHAPTER 1

How it fits together

The environment has three layers, and each one has a single source of truth inside the Tadashi folder.

The first layer is *tools*. `install.ps1` installs about thirty command-line programs through winget and uv, with Node managed by fnm rather than a system-wide installer. It checks before it installs, so running it twice changes nothing.

The second layer is *standing orders*. `claude/CLAUDE.md` is copied to `~/.claude/CLAUDE.md`, the one file Claude Code reads at the start of every session in every project. It sets the bar for finished work, the tools to reach for, the design rules and the writing rules. A new project inherits all of it before the first message.

The third layer is *skills*. Each skill is a folder with instructions, and sometimes scripts, that Claude loads only when the task calls for it. Eight were written for this environment, thirteen were chosen from public repositories after a full read, and a library of more than three hundred more sits on disk for later.

The folder

```
tadashi/
  Tadashi.pdf     this manual
  README.md       the short version
  setup/
    install.ps1   tools and Claude Code
    deploy.ps1    puts everything in place
    library.ps1   mirrors public skill repos
    osint.ps1     the intelligence toolkit
    privacy.ps1   user and machine hardening
    documents.ps1 LaTeX, Office and OCR toolchain
  claude/
    CLAUDE.md     the standing orders
    statusline.py Claude Code status line
    skills/       the active skills, as source
  config/
    starship.toml prompt
    profile.ps1   PowerShell additions
    terminal.json Windows Terminal theme
    editor.json   editor settings
  manual/         the source of this manual
  assets/         the readme art
```

A new machine

Copy the folder, then run three scripts in order. The first installs tools, the second places configuration and skills, and the third mirrors the skill library. The intelligence toolkit, the privacy hardening and the document toolchain have scripts of their own, described in their chapters.

```
powershell -ExecutionPolicy Bypass -File setup\install.ps1
powershell -ExecutionPolicy Bypass -File setup\deploy.ps1
powershell -ExecutionPolicy Bypass -File setup\library.ps1
```

Every file that `deploy.ps1` replaces is copied first to `~/.tadashi-backup`, under a folder named for the date and time of the run. Undoing a deploy means copying those files back.

CHAPTER 2

The command line

Each tool here replaces something slower or vaguer. The table lists what it replaces and the reason it stays. The last chapter describes every tool on the machine in a line or two.

Table 1. The core set and what each one replaced.

TOOL	REPLACES	WHY IT STAYS
rg	grep, findstr	Searches a large repository in milliseconds
fd	dir /s, find	Finds files by name, respects .gitignore
ast-grep	regex refactors	Matches code by syntax tree, not text
jq, yq	manual parsing	Query and edit JSON and YAML in place
sd	sed	Find and replace with sane escaping
bat, eza	cat, dir	Highlighted files, readable listings
zoxide	cd	Jumps to frequent folders by fragment
just	scattered scripts	One file of named project commands
hyperfine	guesswork	Statistically sound benchmarks
uv	pip, venv	Python installs in seconds, isolated
bun, biome	npm, eslint	Fast installs, tests and linting
typst, pandoc	Word, LaTeX	Typesetting and format conversion
gitleaks	hope	Finds secrets before they are pushed

Search by structure

Text search finds strings; structural search finds code. When a refactor touches every call site of a function, `ast-grep` matches the shape of the call regardless of spacing or line breaks, then rewrites it.

```
# every call to fetch_user with two arguments
ast-grep -p 'fetch_user($A, $B)' -l python

# rename the call and swap the arguments
ast-grep -p 'fetch_user($A, $B)' \
  -r 'load_user($B, $A)' -l python -U
```

Measure before you claim

A change that feels faster often is not. `hyperfine` runs each command until the numbers settle and reports the mean with its spread, so a claim about speed comes with evidence.

```
hyperfine --warmup 3 'old-build' 'new-build'
```

The shell

Windows Terminal now opens with a near-black background, a neon palette and JetBrains Mono with the Nerd Font glyphs that the prompt needs. The prompt, drawn by starship, shows the folder, the git branch and its state, the language version when a project declares one, and the duration of any command that ran longer than two seconds. Then it gets out of the way.

PowerShell 7 is the default shell in Windows Terminal and in the editor, which uses the same Nerd Font for its terminal, ligatures in code and a line height of 1.6. Claude Code shows a status line in the same palette: folder, branch, model, how full the context is, cost so far, and lines changed.

The PowerShell additions apply only to interactive windows. `ls` becomes a grouped, icon-aware listing, `ll` adds git status per file, `lt` draws a tree, and `lg` opens lazygit. Scripts and agents get stock PowerShell, so nothing they rely on changes underneath them.

CHAPTER 3

The agents

Claude Code is the only agent on this machine, by choice. It runs in the terminal as `claude` and inside the editor, and both read the same standing orders, skills and memory. One agent with sharp instructions beats several that each need their own setup.

Standing orders

The standing orders in `~/.claude/CLAUDE.md` are short on purpose, because they are read at the start of every session and every line costs attention. They cover six areas:

1. what counts as finished, which is verified, polished and without placeholders;
2. how to move fast, by acting on sensible defaults and running independent work in parallel;
3. which tools to reach for on this machine;
4. how to start anything new, through the genesis skill;
5. the design rules for anything visual;
6. the writing rules for anything a person will read.

A project can add its own `CLAUDE.md` with commands and conventions. It adds to the standing orders and never lowers them.

Memory

Claude keeps a memory folder per project, where it records durable facts: a preference stated once, a decision and its reason, a pointer to an external system. The standing orders hold what is true everywhere, and the project memory holds what is true here. Between the two, a new session starts with the context that used to take ten minutes of explanation.

Working in parallel

Large tasks go faster when independent parts run at once. Claude can send subagents to search a codebase or research a question while it keeps working, and it can run long builds and installs in the background.

CHAPTER 4

Skills

A skill is a folder whose `SKILL.md` tells Claude when it applies and how to do the work. Only the one-line descriptions load at the start of a session; the full instructions load when a task matches. That makes each active skill cheap, but not free, so the active set is kept small.

The active set

Twenty-one skills are active. Eight were written for this environment and thirteen came from public repositories, each read in full first.

Three of the imported ones were edited on the way in. The test-driven skill now fires only for logic and bug fixes, not for styling or copy, so it adds rigour without slowing small changes. The grilling skill lost its emoji and divider lines. The planning skill carries its definition-of-done checklist with it.

The second wave came in with the same care. Anatomy arrives without its prebuilt example pages, which its own build scripts regenerate, so it costs 2 MB instead of 15. Impeccable is a deep design toolkit, but atelier already owns UI work, so impeccable stays quiet until you call it by name. Its engine reports design choices home by default; deploy switches that off. Its two paid features, image generation and live copy editing through an API, need a key this machine never sets, so they cannot spend money. From superpowers, only the three skills that filled real gaps came in. The rest duplicate skills already here, and the full plugin would load itself into every session.

Table 2. Skills active in every session, beyond those that came with Claude.

SKILL	FIRES WHEN	SOURCE
folio	a PDF is wanted	written here
genesis	anything new begins	written here
osint	an investigation starts	written here
authorship	committing or commenting	written here
atelier	any web UI	written here
essentials	before a site ships	written here
vibe	most writing	written here
voice	formal writing	written here
verification-before-completion	about to claim done	superpowers
diagnosing-bugs	something is broken	Matt Pocock
test-driven-development	logic or a bug fix	superpowers
planning-and-task-breakdown	work is too big	Addy Osmani
grilling	a plan needs testing	Matt Pocock
receiving-code-review	feedback arrives	superpowers
build-mcp-server	building MCP servers	Anthropic
karpathy-guidelines	writing or reviewing code	Forrest Chang
using-git-worktrees	work needs isolation	superpowers
finishing-a-development-branch	a branch is done	superpowers
dispatching-parallel-agents	tasks can run at once	superpowers
anatomy	a figure explains an idea	wheresryan22
impeccable	you type `/impeccable`	Paul Bakaus

The library

`library.ps1` keeps shallow mirrors of ten public skill repositories in `~/.claude/skill-library`, 317 skills in all. Nothing there loads on its own. `INDEX.md` lists every skill with a short description, and `CURATION.md` explains which ones were chosen and why.

The mirrors are treated as untrusted. Cloning runs with git hooks disabled, without submodules or large-file downloads, and nothing inside a mirror is ever executed. Every skill in the active set was read in full before it was copied in.

Some skills belong to one project rather than to every session. Vercel's React performance rules suit a Next.js codebase, and Cloudflare's security audit suits a full review of a service. To use one, copy its folder into that project's `.claude/skills` directory.

CHAPTER 5

Documents with folio

Folio produces one kind of PDF, and produces it well: a white page set in EB Garamond, with wide margins, real small caps and old-style figures. It has a title page and a contents page, and each chapter starts on a new page. Tables, footnotes and section breaks use space instead of lines, and nothing on the page is ruled, boxed or shaded.

Writing the source

Write Markdown. One # begins a chapter, two begin a section, three begin a subsection. Front matter sets the title page.

```
---
title: Quarterly review
subtitle: Infrastructure, July to September
author: Platform team
date: October 2026
---

# Where the money went

Compute spend fell by a fifth after the
move to reserved capacity.
```

Building

Ask Claude for a PDF and the skill runs on its own. To run it by hand, check the prose and then build with page previews.

```
python ~/.claude/skills/folio/scripts/folio.py check doc.md
python ~/.claude/skills/folio/scripts/folio.py build doc.md \
  --png pages
```

The check is a linter for machine-written prose. It fails the build on phrases that mark text as generated, on emoji, on stacks of bold-labelled bullets and on heavy

use of the em dash. It warns about filler words. A document that passes still deserves a careful read, but it will not read like a template.

The previews matter as much as the PDF. Folio's instructions require Claude to look at every page image before calling the work finished: no stranded headings, no loose lines, no table wider than the text, no code that wraps.

LaTeX and Office files

When a journal or university template demands real LaTeX source, Tectonic compiles it. It fetches the packages a document needs on its own and never stops to ask. Word, PowerPoint and Excel work goes through their own skills, backed by LibreOffice, QPDF, Tesseract and the Python and Node libraries they call. `setup\documents.ps1` installs all of it and proves each piece works, so no session ever installs a tool halfway through a task.

Writing Typst directly

For layouts that Markdown cannot express, write Typst and import the template. The same build command accepts a `.typ` file.

```
#import "@local/folio:1.0.0": folio
#show: folio.with(title: [Field notes], date: [2026])

= First chapter
```

CHAPTER 6

Starting something new

The genesis skill fires whenever a new project begins. It picks a stack from the request, runs the official generator, and adds the quality gates before the first feature exists.

For Python that means uv, ruff, basedpyright and pytest. For TypeScript it means bun, strict compiler settings, biome and bun's test runner. Web applications get Vite or Next.js with Tailwind. Every project receives an `.editorconfig`, line-ending rules, a short README, a project `CLAUDE.md`, and a `justfile` with the same four commands.

```
just dev     # run it
just check   # format, lint, types, tests
just fmt     # fix what can be fixed
just test    # tests alone
```

The project is not handed over until `just check` passes. Quality gates that exist from the first minute stay green; gates added later start red and tend to stay that way.

CHAPTER 7

Intelligence

Open-source intelligence answers questions from public sources: who runs a domain, which subdomains a company has exposed, where a handle appears, what a photograph's metadata says. `osint.ps1` installs a toolkit for it, and the osint skill tells Claude how to use it with method and within limits.

The toolkit

Table 3. The intelligence toolkit by kind of target.

TARGET	TOOLS
Domains	subfinder, theHarvester, amass, dnsx, dnstwist
Hosts	httpx-toolkit, Shodan and Censys clients
People and accounts	sherlock, maigret, holehe, socialscan
Media	exiftool, yt-dlp, gallery-dl, ffmpeg
Correlation	SpiderFoot, in a local container

Every tool is isolated. The Python tools each live in their own uv environment, so none can break another. The ProjectDiscovery binaries come from official releases and are installed only after their SHA-256 checksums match the published ones. SpiderFoot runs in Docker and listens on this machine alone; typing `spiderfoot` starts it and opens the interface.

Method

An investigation starts with one sentence that states the question, because the question decides when to stop. Collection is passive before it is active and broad before it is deep, and each finding pivots to the next: a domain gives addresses, an

address gives usernames, a username gives images. Nothing counts as a finding until two independent sources agree, and every source is recorded with its address, the time it was read and an archived copy.

The limits are part of the method. Only public sources are used. No account is ever accessed, no leaked credential is ever tried, and private individuals are in scope only for a clear and lawful purpose, such as auditing your own exposure. Active steps, such as probing hosts, are reserved for systems you own or have written permission to test.

Each case gets a folder with the question, the raw output, an evidence log and a report, and the report is typeset with folio.

CHAPTER 8

Privacy

Every tool added for privacy meets one bar: free, open source and independently audited, with the recommendations of the Privacy Guides project as the reference. Where nothing met that bar, nothing was installed.

What changed

`privacy.ps1` works in two halves. The first runs as you and needs no elevation. It turns off the advertising identifier, tailored experiences, suggested content and silently installed apps. It keeps speech, typing and inking data on the device. It shows file extensions and hidden files, which exposes tricks such as `invoice.pdf.exe`, and it keeps recent files out of Quick Access and web results out of Start.

The second half runs elevated. It lowers diagnostic data to the minimum Windows allows, turns off activity history, Copilot and Remote Assistance, and sets Defender to block unwanted applications and known malicious domains. The firewall is on for every network profile and refuses inbound connections by default. DNS goes to Quad9 over HTTPS, with no fallback to plain DNS, so neither the network owner nor the provider sees which sites you look up.

Every registry key is exported before it changes. Undoing the script means importing the files in `~/.tadashi-backup`.

Trusted applications

Table 4. Installed because each one meets the trust bar.

APPLICATION	PURPOSE
Bitwarden	Passwords and secrets, with the `bw` command line
Proton VPN Free	Audited fallback VPN, installed but idle
Mullvad Browser	Hardened browser for research and investigations
VeraCrypt	Encrypted containers for sensitive files

Cloudflare WARP is the VPN on this laptop, by choice. While it is connected, traffic and DNS travel encrypted to Cloudflare; when it is off, Quad9 over HTTPS takes over, so lookups never leave in the clear. Proton VPN Free stays installed as a fallback. Run one VPN at a time, never both.

Watching the machine

Protection is half the job; the other half is seeing what the machine does. Microsoft's Sysinternals tools show it without guesswork. After installing anything, open `autoruns` and confirm that nothing new starts with Windows unless you expect it. When the fan spins or the network is busy for no reason, `procexp` and `tcpview` name the process responsible and the address it is talking to. Before running an unfamiliar program, `sigcheck` confirms who signed it.

Disk encryption

An unencrypted drive hands every file to whoever holds the laptop, including SSH keys and signed-in sessions. `privacy.ps1 -Machine` reports the state of the system drive; if it warns, fix that before anything else. Device encryption needs a TPM 2.0 chip and modern standby, which most recent laptops have.

Windows Home's device encryption needs a Microsoft account once, to hold the recovery key. Sign in, then turn it on under Settings, Privacy and security, Device encryption. When it finishes, save the key yourself from an elevated PowerShell:

```
manage-bde -protectors -get C:
```

Store the recovery password in Bitwarden and on paper. Then delete the copy at `account.microsoft.com/devices/recoverykey`, and switch back to a local

account if you prefer. The disk stays encrypted, and the only copies of the key are yours.

VeraCrypt can encrypt the system drive without involving Microsoft at all, but its boot loader needs Secure Boot set up for it, and Windows feature updates can clash with it. Device encryption is the better choice on most laptops.

CHAPTER 9

The arsenal

What follows are recommendations beyond what is installed: typefaces, references and habits that keep work fast and finished.

Typefaces

A small set of free typefaces covers almost everything. For reading, EB Garamond suits books and reports, Newsreader suits editorial pages on screen, and Source Serif 4 suits dense technical prose. For interfaces, Geist is precise and neutral, Inter Tight is a compact cut of the most legible interface face, and Instrument Sans has more character. For display sizes, Instrument Serif and Fraunces give a headline presence that a text face cannot. For code, JetBrains Mono and Geist Mono are both excellent and free.

For code with more character, Commit Mono is tuned for calm reading and GitHub's Monaspace family offers five related faces that can mix in one editor. Every typeface named here is free.

Pair at most two families. A serif for reading with a grotesque for interface labels is the safest pairing that still looks deliberate.

Interfaces

For web interfaces, the fastest route to a polished result is Tailwind, with components from shadcn/ui, which are built on Radix primitives. Lucide supplies line icons, which keeps emoji out of the interface, and Motion handles animation. Keep transitions between 150 and 250 milliseconds with an ease-out curve, and animate only what helps the eye follow a change.

The design rules in the standing orders reduce to one idea: hierarchy comes from size, weight and space, not from boxes and lines. A page with fewer borders and more room nearly always looks more expensive.

References worth owning

Practical Typography by Matthew Butterick is free online and fixes most typographic mistakes in an afternoon. *Refactoring UI* by Adam Wathan and Steve Schoger turns design intuition into rules a developer can apply. Robert Bringhurst's *The Elements of Typographic Style* is the standard reference for print. The books from Stripe Press show how far plain, careful typography can go.

Habits

Speed comes less from typing and more from removing waiting. Run long work in the background. Give each project one command that checks everything, and run it before every commit. Measure before claiming a gain. Ask for one recommendation instead of three options. When a task repeats a third time, turn it into a skill.

CHAPTER 10

Keeping it sharp

The environment drifts unless it is refreshed. A monthly pass takes a few minutes.

```
winget upgrade --all
uv tool upgrade --all
subfinder -up; dnsx -up; httpx-toolkit -up
powershell -File setup\library.ps1
powershell -File setup\deploy.ps1
```

Edit the standing orders and skills inside the Tadashi folder, never in `~/.claude` directly, then deploy. The folder stays the single source of truth, and any machine can be brought back to it.

When a skill in the library looks useful, read its `SKILL.md` in full before copying it into the active set. A skill is a set of instructions that runs with your permissions. It deserves the same scrutiny as a dependency.

CHAPTER 11

Tool reference

Every tool on this machine, with what it does and how it helps, in a line or two. Commands are set in monospace; applications by name.

Finding things

`rg`
: Searches the contents of every file in a project in milliseconds, skipping whatever `.gitignore` excludes. Use it to find every place a name, string or error message appears.

`fd`
: Finds files and folders by name, faster and with simpler syntax than Windows search. Use it when you know roughly what a file is called but not where it lives.

`fzf`
: Filters any list interactively as you type. Pipe files, branches or history into it and pick the one you want in a keystroke or two.

`zoxide`
: Learns the folders you visit and jumps to them from a fragment of the name: `z tadashi` from anywhere. It removes most `cd` typing.

`eza`
: A clearer directory listing with folders first, icons and git status. In an interactive shell it answers to `ls`, `ll` (with git status) and `lt` (as a tree).

`bat`
: Prints a file with syntax highlighting and line numbers; the shell shortcut is `b`. Reading code in the terminal stops being a chore.

`dust`
: Shows which folders take up disk space, as a tree sorted by size. Use it when the drive fills up and you need to know why.

`glow`
: Renders Markdown in the terminal with headings, lists and code formatted. Read a README properly without leaving the shell.

Data and text

`jq`
: Queries and reshapes JSON from the command line. Pull one field out of an API response, or filter a large file, in a single line.

`yq`
: Does the same for YAML, and reads JSON, TOML and XML too. It edits configuration files in place without breaking their structure.

`sd`
: Find and replace across files with ordinary regular expressions. It does what `sed` does, without the escaping puzzles.

`ast-grep`
: Searches and rewrites code by its syntax tree instead of its text. One command can rename a function call at every site, however it is formatted.

Code quality

`ruff`
: Lints and formats Python, replacing flake8, black and isort with one tool that runs in milliseconds. Style arguments end, and real mistakes surface before the code runs.

`biome`
: Lints and formats JavaScript, TypeScript and JSON, replacing ESLint and Prettier. One fast tool keeps a web codebase consistent.

basedpyright
: Type-checks Python. Genesis adds it to each Python project, where it catches whole classes of bugs before the code runs.

`gitleaks`
: Scans a repository and its history for passwords, tokens and keys. Run it before every push so a secret never reaches GitHub.

Building and measuring

`just`
Runs named project commands from a `justfile`: `just dev`, `just check`, `just fmt`. Every project starts and checks itself the same way.

`hyperfine`
Benchmarks commands, running each until the timings settle and reporting the mean and spread. A claim that something got faster arrives with proof.

`tokei`
Counts lines of code, comments and blanks by language. Use it to size up an unfamiliar codebase in seconds.

`btop4win`
A live dashboard of processor, memory, disk, network and processes. Find what is slowing the machine down at a glance.

Web

`lighthouse`
Scores a page for speed, accessibility and SEO the way Google measures them. Every site is held to 100 before it ships.

`svgo`
Strips SVG files down to what renders. Icons and illustrations load in a fraction of the bytes.

`pyftsubset`
Cuts a font down to the characters a site uses and saves it as WOFF2. Pages show their real typeface without waiting.

Playwright
Drives a real Chromium to load pages, click through flows and take screenshots at any width. Design work is checked by looking, not hoping.

Git and GitHub

`git`
Version control for every project, configured with modern defaults: rebase on pull, conflict views that show the original, and remembered resolutions.

`gh`
: GitHub from the terminal: pull requests, issues, releases and workflow runs. Open and review a pull request without touching the browser.

`delta`
: Shows git diffs with syntax highlighting and line numbers. Changes are easier to read, so reviews are faster and catch more.

`lazygit`
: A full git interface inside the terminal; `lg` opens it. Stage single lines, rewrite history and resolve conflicts with a few keys.

Languages and runtimes

`uv`
: Installs Python versions, project dependencies and command-line tools, each isolated, ten to a hundred times faster than pip. Python setups stop breaking each other.

`fnm`
: Switches Node versions automatically to whatever a project asks for. Each project runs on the Node it was built for.

`bun`
: A JavaScript runtime, package manager, test runner and bundler in one fast binary. JavaScript projects install and test in a fraction of the usual time.

Python
: Python 3.12 is the system interpreter, and uv provides any other version a project needs without touching it.

PowerShell 7
: The modern PowerShell and the default shell everywhere, with history-based suggestions and faster startup than the built-in Windows PowerShell.

Docker Desktop
: Runs software in sealed containers. Here it keeps SpiderFoot apart from the rest of the system.

Documents

`typst`
: The typesetting engine behind folio. It turns source into a finished PDF in about a second, with real typography.

`pandoc`
: Converts between document formats: Markdown, Word, HTML, Typst and dozens more. Bring any document into folio, or send one out as Word.

`tectonic`
: A complete LaTeX engine in one file that downloads only the packages a document uses. Compile any `.tex` template with `tectonic -X compile file.tex`.

LibreOffice
: Opens, converts and renders Word, PowerPoint and Excel files without Microsoft Office. It turns any of them into PDF for checking or sending.

`qpdf`
: Checks, repairs, merges and splits PDFs without changing their content. A damaged PDF opens again.

Tesseract
: Reads the text out of scanned pages and images. Scans become searchable and quotable.

`markitdown`
: Turns Word, PowerPoint, Excel and PDF files into clean Markdown. Any office document becomes material for folio or for analysis.

`pdftoppm`, **`pdffonts`**, **`pdfinfo`**
: PDF inspection from poppler: render pages as images, list embedded fonts, read metadata. Check that a PDF is right before it leaves the machine.

Shell and terminal

starship
: Draws the prompt: folder, git branch and state, language version, and a timer for slow commands. You always know where you are and what state the repository is in.

JetBrains Mono Nerd Font
: The coding typeface in Terminal and the editor, with the extra symbols the prompt uses. Code is easier to read and the prompt renders correctly.

Windows Terminal
: Runs with the Tadashi Neon theme, generous padding and PowerShell 7 by default, so the terminal is pleasant to spend a day in.

The agent

Claude Code
: The coding agent, in the terminal as `claude` and inside the editor, following the standing orders, skills and memory. Routine work meets the standard without restating it.

Status line
: A line under the Claude Code prompt showing folder, branch, model, context used, cost and lines changed. You see the state of a session without asking.

Chimes
: A soft calendar chime when Claude finishes a turn and a message chime when it needs you, both from sounds Windows already ships. Long tasks can run in another window while you work.

Skills

folio
: Turns Markdown into a book-grade PDF, linting the prose first and checking every page after. Documents look finished the first time.

genesis
: Sets up any new project with formatting, linting, types, tests and one-command checks before the first feature. Quality is built in from minute one.

osint
: Guides an investigation from question to evidence-backed report, with the toolkit and its limits. Findings are verified, sourced and repeatable.

atelier
Designs and builds web frontends at award level, from direction and palette to glass navigation, motion, speed and SEO. Sites look considered and load instantly.

essentials
The minimal, non-negotiable pieces every site ships, scaled to its type and feel. Nothing obvious is missing on launch day.

vibe
The everyday voice: warm and kind, with real energy and a little swagger earned by specifics. Words people remember without feeling sold to.

voice
The formal register for applications, professors, employers and official documents: warm, kind, professional and humble.

authorship
Keeps every commit, pull request, comment and doc authored by you alone, written brief and plain. Your history reads like one careful engineer wrote it.

verification-before-completion
Requires fresh evidence before anything is called done or passing. Nothing is reported finished that was not seen working.

diagnosing-bugs
Builds a reliable way to reproduce a bug, then tests ranked hypotheses against it. Hard bugs get fixed at the cause, not patched at the symptom.

test-driven-development
Writes the failing test before the fix for logic and bug work. Every fix arrives with proof, and the bug cannot quietly return.

planning-and-task-breakdown
Splits large work into small, verifiable slices with acceptance criteria. Big projects move in steps that can each be checked.

grilling
Questions a plan round by round, each question with a recommended answer. Weak spots surface before any code is written.

receiving-code-review
Checks each review comment against the code before acting on it. Good feedback is applied, and wrong feedback is answered with evidence.

build-mcp-server
: Anthropic's guide to building MCP servers that connect Claude to other systems. New integrations start from the right design.

karpathy-guidelines
: Four rules from Andrej Karpathy's notes on how language models go wrong in code: think first, keep it simple, change only what was asked, define what done looks like. Diffs stay small and every line traces back to the request.

using-git-worktrees
: Gives a feature its own working copy before the work starts. Experiments never trample the branch you are standing on.

finishing-a-development-branch
: Walks a finished branch through tests, then a clear choice to merge, open a pull request, keep it or discard it. Nothing is left half-merged.

dispatching-parallel-agents
: Sends independent problems to separate agents at once, each with exactly the context it needs. Three investigations take the time of one.

anatomy
: Explains an idea by building it as a working object: a lock, a gauge, a plotter, drawn in isometric SVG with live parts and an audit that checks every joint. Diagrams people stop scrolling for.

impeccable
: A design director's toolkit behind one command, `/impeccable`, with audit, critique, polish, typeset, animate and about twenty more moves. Good interfaces get the last ten percent.

Intelligence

`sherlock`
: Checks hundreds of websites for a username. Map where a handle is used in under a minute.

`maigret`
: A deeper username search across thousands of sites that pulls profile details and writes an HTML or PDF report. Build a fuller picture of an online identity from public pages.

holehe
Finds which services an email address is registered with, through their public sign-up and recovery pages, without notifying the address owner. Use it to audit your own exposure.

socialscan
Checks whether a username or email is taken on major platforms, quickly and accurately. Confirm presence or availability before digging deeper.

theHarvester
Collects email addresses, subdomains, hosts and IP addresses for a domain from public sources such as search engines and certificate logs. It is the first pass on any organisation.

subfinder
Discovers a domain's subdomains passively from dozens of public sources, without touching the target. It shows how large an organisation's surface really is.

amass
OWASP's mapper of a domain's infrastructure: subdomains, networks and how they connect. Use it when a subdomain list needs depth and context.

dnsx
Resolves thousands of hostnames quickly and keeps the ones that answer. It turns a long candidate list into the hosts that exist.

dnstwist
Generates look-alike domains through typos, swapped letters and similar characters, then checks which are registered. Spot phishing and impersonation domains early.

httpx-toolkit
Probes hosts for live web servers and reports titles, status codes and technologies. This step is active, so run it only against systems you own or may test.

shodan
Searches Shodan's index of internet-facing devices and services; a free account provides the key. See what a network exposes without scanning it yourself.

censys
Searches certificates and hosts across the internet, with a free account key. Find infrastructure linked to an organisation through its certificates.

`exiftool`
: Reads and edits metadata in photos, PDFs and documents: GPS position, camera, author, software and dates. A single file can reveal where and how it was made.

`yt-dlp`
: Downloads video and its metadata from YouTube and over a thousand other sites. Preserve evidence before it is deleted.

`gallery-dl`
: Archives images and posts from galleries and social sites. Capture a public profile as it stands today.

`ffmpeg`
: Converts, cuts and inspects audio and video, and pulls single frames out of footage. It also does the merging behind yt-dlp.

SpiderFoot
: Runs hundreds of automated modules from one starting point in a local web interface; type `spiderfoot`. It widens an investigation fast.

Privacy and security

Bitwarden
: An open-source, audited password manager, with `bw` for scripts. Every password stays unique and strong, and secrets never sit in code.

Mullvad Browser
: A browser built with the Tor Project that makes every user look alike to trackers. Research and investigate without being profiled.

Cloudflare WARP
: The VPN in daily use, encrypting traffic and DNS on untrusted networks.

Proton VPN Free
: An audited no-logs VPN kept as a fallback; never run it alongside WARP.

VeraCrypt
: Creates encrypted containers that open only with your password. Keep case files and sensitive documents unreadable to anyone else.

Quad9
: Encrypted DNS that refuses known malicious domains, so lookups stay private and many phishing links never load.

`autoruns`
: Lists everything that starts with Windows: services, drivers, scheduled tasks, browser add-ons. Anything that sneaks in to run at startup has nowhere to hide.

`procexp`
: Process Explorer, a far deeper Task Manager that shows which program started which, what each has open, and whether it is signed. Find the real culprit behind a slow or noisy machine.

`tcpview`
: Shows every network connection live, with the program that owns it. See at once which app is talking to the internet, and to where.

`sigcheck`
: Verifies a file's digital signature and version details. Confirm a download is genuine before running it.

Microsoft Defender and firewall
: Blocks unwanted apps and known malicious sites, while the firewall refuses inbound connections on every network.